

DAFFODIL INTERNATIONAL UNIVERSITY
FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING
Course: SE-231 (Software System Analysis & Design / Capstone Project 2)

CAPSTONE PROJECT PROPOSAL
PART 1: PROJECT PLANNING AND DEFINITION

RetailSync: Centralized Super Shop Warehouse Management System

Real-Time POS Synchronization, Dynamic Putaway, FEFO Batch Tracking, and AI Demand Forecasting

PROJECT SUBMISSION CREDENTIALS

Project Team Members & Exact Student IDs:

- Raisul Islam Likhon (Project Lead & System Architect) — ID: 251-35-508
- Shottobroto Dey (Database & Systems Engineer) — ID: 251-35-017
- Golam Husnain Papon (Full-Stack & QA/DevOps Engineer) — ID: 251-35-529

Batch & Section: 44th Batch, Section: SWE-44D (B.Sc. in Software Engineering)

Department & Faculty: Department of Software Engineering, Faculty of FSIT

Institution: Daffodil International University (DIU), Dhaka, Bangladesh

Academic Supervisor: MD. Selim Reza (Assistant Professor), Dept. of SWE

Course & Term: SE-231 | Term: Fall 2026

Executive Summary & Project Abstract

In modern supermarket chains across Bangladesh such as Shwapno, Agora, Meena Bazar, and Unimart—inventory management is frequently split between two disconnected operational worlds: the front-of-house checkout registers and the back-of-house distribution warehouses. When a customer purchases a carton of milk or a bottle of edible oil at the billing counter, that transaction is recorded in a siloed Point of Sale (POS) database. Back-office warehouse staff often only discover stock depletion hours or even days later through delayed manual tallies or periodic spreadsheets. This structural disconnect leads directly to severe operational losses: perishable goods expire unseen on rear shelves, unrecorded breakages cause 'phantom inventory', and popular grocery staples run out during evening and festival rushes.

RetailSync was designed to solve this exact industry challenge. It is a centralized, high-performance Warehouse Management System (WMS) engineered specifically for the fast-paced grocery retail sector in Bangladesh. The system unifies central distribution warehouses and frontline checkout counters into a single, real-time relational core. By enforcing strict Third Normal Form (3NF) database constraints, pessimistic row-level locking on checkout deductions, directed spatial putaway, automated First-Expired, First-Out (FEFO) batch rotation, and machine-learning demand forecasting (CatBoost with native festival calendar awareness; LightGBM deferred to roadmap), RetailSync directly tackles the three largest profit leaks in supermarket operations: (1) perishable food spoilage (15% to 22% annual loss), (2) unrecorded inventory shrinkage (1.8% to 2.4% write-offs), and (3) peak-hour stockouts during festival surges (7.5% to 11.2% lost sales).

From an engineering perspective, RetailSync is implemented as a 4-tier cyber-physical architecture combining a Next.js 14 Progressive Web Application (PWA), an asynchronous Python 3.11+ FastAPI backend, a normalized PostgreSQL 16 relational ledger, and an in-memory Redis 7 caching tier. Rather than demanding expensive industrial scanning hardware (such as 60,000 BDT Zebra terminals), RetailSync runs seamlessly on standard 12,000 BDT consumer Android smartphones paired with 3,800 BDT Bluetooth barcode trigger grips, reducing frontline hardware deployment costs by nearly 90%.

CORE CAPSTONE THESIS & QUANTIFIABLE SLO TARGETS

Primary Research Question: How can centralized relational concurrency, machine-learning demand forecasting, and automated FEFO prioritization eliminate retail stockouts and perishable spoilage in high-density grocery operations?

Key Engineering SLO Target: Sub-2.0s POS checkout deduction latency ($p_{95} \leq 800ms$) across 10 concurrent branch registers with exactly zero deadlocks and zero overselling.

Methodology & Validation: 14-Week Agile Scrum Lifecycle delivering an incremental, Dockerized production-grade MVP across 6 sprints, validated against 4 stress-injected supermarket scenarios.

Document Architecture & Table of Contents

This project proposal is structured into ten cohesive chapters providing complete planning, empirical grounding, and architectural blueprints for RetailSync in accordance with DIU Software Engineering Capstone standards:

Section	Title	Summary Scope
Section 1	Industry Background & Problem Statement	Operational context, profit leaks, grounded value taxonomy, and problem-to-feature mapping.
Section 2	Project Objectives & Scope Boundaries	Quantitative SMART goals (O-01 to O-05), pilot constraints, and 4-quadrant scope boundaries.
Section 3	Target Personas & Operational Workflows	Four core supermarket personas (Manager, Cashier, Operator, Buyer) and end-to-end floor journeys.
Section 4	Core Functional Modules & AI Replenishment Engine	Eleven modular domain services with in-depth spotlight on Module M-07 (CatBoost & Greasley DSS).
Section 5	Non-Functional Requirements & Performance SLOs	Latency SLOs, throughput benchmarks, ACID concurrency guarantees, and Food Safety Act compliance.
Section 6	System Architecture & Technical Design	4-tier architecture, process data flow, AI ML pipeline, PostgreSQL 16 3NF schema, and row-locking.
Section 7	Curated Technology Stack & Hardware Strategy	Full-stack technology matrix (Next.js, FastAPI, PostgreSQL, Redis) and frugal smartphone scanner model.
Section 8	Development Methodology: Agile Scrum Framework	Scrum rationale, 14-week milestone Gantt, 4 stress-injected supermarket scenarios, and Definition of Done.
Section 9	Resource Allocation, Budget & Risk Management	RACI matrix, realistic student hardware budget (< 35,000 BDT), and local operational risk mitigations.
Section 10	Verification, ROI Impact, References & Conclusion	Multi-tier testing strategy, 2.8-month financial payback model, academic literature citations, and summary.

1. Industry Background & Problem Statement

1.1 The Modern Grocery Retail Landscape in Bangladesh

The supermarket and organized grocery retail sector in Bangladesh is expanding rapidly. Driven by rapid urbanization (approximately 65.2 million urban citizens, representing 39.7% of the total population according to BBS Census 2022), rising disposable incomes, and dual-earner households, consumers in metropolitan centers like Dhaka and Chattogram increasingly rely on super shops for their daily groceries. Well-known chains such as Shwapno (operating over 450 outlets), Agora, Meena Bazar, Unimart, and Daily Shopping handle tens of thousands of fast-moving consumer goods (FMCG) and fresh produce items daily.

However, behind the clean checkout counters and barcode scanners of modern storefronts lies a fragile back-office supply chain. While checkout billing has modernized, warehouse stock management, pallet putaway, batch rotation, and store replenishment remain reliant on manual paper manifests, clipboard logs, and fragmented Excel sheets. When a supplier truck delivers 200 cartons of milk to a central warehouse in Tejgaon, workers often check items against a paper purchase order, hand-write batch numbers, and store pallets in whatever aisle has open space. Because there is no real-time synchronization with retail POS terminals, stock data quickly drifts out of sync.

1.2 The Three Critical Profit Leaks in Super Shop Operations

- **1. Perishable Food & Dairy Spoilage:** Supermarket chains lose between 15% and 22% of perishable food (pasteurized milk, yoghurt, poultry, fresh fruits, and chilled items) annually due to lack of batch-level expiration tracking. When floor staff restock retail shelves, they naturally place incoming pallets at the front because it is physically easier, pushing older batches to the rear where they spoil unnoticed. Selling expired food exposes the supermarket to heavy fines and closure under the Bangladesh Food Safety Act 2013.
- **2. Phantom Inventory & Operational Shrinkage:** An average of 1.8% to 2.4% of total inventory value disappears every year through unrecorded breakages, packaging tears, internal pilferage, and inaccurate delivery counts. Because physical audits occur only once a quarter, the computer system reports items as 'in stock' when the physical shelf is actually empty a phenomenon known as 'phantom inventory'.
- **3. Peak-Hour Stockouts & POS Freezing:** During high-volume shopping periods such as the holy month of Ramadan, Eid-ul-Fitr, Shab-e-Barat, and Friday evening rushes fast-moving staples like edible oil, sugar, and milk sell out in minutes. Because POS checkout counters do not deduct warehouse stock atomically, replenishment alerts lag by hours, leading to empty shelves and an estimated 7.5% to 11.2% in unrealized retail sales from frustrated customer walkouts.

1.3 Grounding of Operational Claims: Literature Baselines, Engineering SLOs & Simulation Assumptions

During capstone evaluation and defense, project proposals are often criticized if performance numbers and problem statistics appear arbitrary or unsubstantiated. To establish complete transparency and academic rigor, every figure cited in this proposal is explicitly classified into one of three epistemological categories: (1) Literature & Industry Case Studies published by verified organizations, (2) Empirical Engineering SLOs that will be benchmarked directly through software testing, and (3) Modeled Simulation Assumptions projected for controlled pilot validation:

Metric / Parameter	Claimed Baseline / Target	Formal Classification	Empirical Grounding / Verification Protocol
Perishable Spoilage Rate	15% to 22% annual dairy/produce loss	Industry Case Study & Literature Baseline	Documented in Bangladesh Supermarket Owners Association (BSOA) field reports

Metric / Parameter	Claimed Baseline / Target	Formal Classification	Empirical Grounding / Verification Protocol
			and FAO South Asia Post-Harvest Retail Loss assessments in urban grocery chains.
Phantom Shrinkage Write-offs	1.8% to 2.4% unexplained inventory loss	Industry Benchmark Baseline	Aligned with National Retail Security Survey (NRSS) supermarket shrinkage baselines adapted for un-barcoded local FMCG supply chains in Dhaka.
Peak-Hour Stockout Losses	7.5% to 11.2% lost retail revenue	Industry Benchmark Baseline	Derived from IHL Group retail out-of-stock studies during festive demand spikes (Ramadan, Eid-ul-Fitr, weekend rushes in Dhaka super shops).
POS Checkout Scan Latency	$\leq 2.0s$ (p95 $\leq 800ms$, p99 $\leq 1.5s$)	Empirical Engineering SLO	Testable engineering SLA to be verified empirically via Locust multi-threaded load tests executing atomic row-level locks on PostgreSQL 16.
Barcode Decode Latency	≤ 350 ms visual confirmation	Empirical Hardware SLO	Verified on physical consumer Android smartphone (Chrome PWA) paired with Bluetooth HID trigger scanner using ZXing/Html5-QRCode.
Post-Implementation Spoilage	Reduced from 22% down to $< 6\%$	Modeled Simulation Assumption	Mathematical pilot projection assuming 100% strict FEFO compliance and automated quarantine threshold (≤ 3 days) eliminating rear-of-bin rotting.
Stockout Frequency Reduction	$> 60\%$ reduction during surges	Modeled Simulation Assumption	Projected outcome of replacing static monthly manual reorders with automated 7-day rolling AI demand forecasting and Greasley dynamic safety stock.
Terminal Hardware Cost	$< 16,000$ BDT per station (90% savings)	Empirical Budget Benchmark	Direct market quotation pairing consumer Android smartphone (12,000 BDT) with Bluetooth trigger grip (3,800 BDT) vs 60,000 BDT Zebra TC52.
Offline Queue Resilience	Buffer ≥ 200 sales with 0 loss	Empirical Software SLO	Verified by simulating Wi-Fi disconnection on PWA client, storing sales in IndexedDB, and executing bulk idempotent sync upon reconnect.

METHODOLOGICAL GROUNDING: AVOIDING UNSUBSTANTIATED PROPOSAL CLAIMS

- *Literature Baselines* (15-22% spoilage, 1.8-2.4% shrinkage, 7.5-11.2% stockouts) are derived directly from published Bangladesh Supermarket Owners Association (BSOA) field reports and FAO South Asia Post-Harvest loss assessments.
- *Engineering SLOs* (sub-2.0s POS deduction, p95 $\leq 800ms$, sub-350ms barcode decode) are empirically verifiable SLAs enforced by automated Locust stress test suites and physical Bluetooth scanner profilers.
- *Target Improvements* ($< 6\%$ spoilage, $> 60\%$ stockout reduction) represent mathematically modeled pilot projections achieved through strict FEFO algorithmic quarantine and AI-driven dynamic safety stock.

1.4 Operational Gaps in Current Commercial Solutions

Existing commercial software solutions fail Bangladeshi supermarket operators across three key areas: (1) Prohibitive Cost: Enterprise ERPs like SAP, Oracle NetSuite, and Odoo Enterprise require millions of BDT in annual licensing and mandate 60,000 BDT industrial handheld terminals; (2) Batch Blindness: Standard retail POS software (e.g., PrismPOS, Tally) tracks aggregate SKU counts without knowing which specific batch is expiring first; and (3) Cloud Fragility: Pure cloud systems freeze completely when local broadband connections drop, halting checkout counters.

1.5 Problem-to-Feature Mapping

ID	Real-World Problem	Operational Impact	RetailSync Solution
P-01	Data Isolation	Sales data lives in the POS; warehouse data lives on clipboards.	Centralized SQL Database serving both POS and Warehouse APIs.
P-02	Undetected Stockouts	High-demand items run out on shelves before restock is ordered.	Automated Reorder Point (ROP) triggers sending alerts to managers.
P-03	Perishable Waste	Lack of batch tracking leads to expired goods.	Strict FIFO/FEFO logic and expiry-date dashboard highlights.
P-04	Supplier Delays	Manual PO generation is slow and error-prone.	Automated PO generation based on low-stock metrics.
P-05	Internal Shrinkage	Untraceable manual stock adjustments.	Role-Based Access Control (RBAC) with immutable audit logging.

2. Project Objectives & Scope Boundaries

2.1 Upgraded SMART Project Objectives

To ensure our software engineering implementation is testable and defense-ready, we formulated five SMART objectives (Specific, Measurable, Attainable, Relevant, and Time-bound). Each objective ties directly to a verifiable test gate:

ID	Objective Domain	SMART Target & Quantitative Metric	Measurement Protocol & Verification Gate	Sprint Target
O-01	Sub-Second POS Concurrency & ACID Integrity	Execute atomic stock deductions from retail cash registers via PostgreSQL row-level locks (SELECT ... FOR UPDATE SKIP LOCKED), achieving p95 latency $\leq 800\text{ms}$ and p99 $\leq 1.5\text{s}$ with exactly zero deadlocks and zero negative balances across 10 concurrent registers.	Automated Locust multi-threaded load test simulating 10 concurrent cashiers competing for the last inventory batch.	Sprint 4 (Weeks 7–8)
O-02	Automated FEFO Priority & Food Safety Quarantine	Enforce database-level First-Expired, First-Out allocation ensuring 100% of store replenishment orders pick earliest expiring batches; automatically quarantine stock reaching ≤ 3 days to expiry via scheduled daily 02:00 BST Celery sweep, reducing spoilage to $< 6\%$ in pilot simulations.	Automated Pytest batch sorting assertions and Celery scheduled quarantine state transition triggers.	Sprint 3 (Weeks 5–6)
O-03	AI-Driven Demand Forecasting & Dynamic Replenishment	Deploy a CatBoost Time-Series Forecasting engine with native categorical festival embeddings (Ramadan, Eid, paydays) achieving MAPE $\leq 15\%$ on high-velocity FMCG items; dynamically compute Reorder Points (ROP) using Greasley's Safety Stock (LightGBM deferred to roadmap).	Backtesting against 12-month FMCG sales series with walk-forward validation and automated draft PO comparison.	Sprint 5 (Weeks 9–10)
O-04	Frugal Hardware Architecture & Sub-350ms Scanning	Engineer a mobile Progressive Web Application (PWA) running on consumer Android smartphones paired with Bluetooth HID trigger grips ($< 4,000$ BDT), achieving barcode decode-to-render latency $\leq 350\text{ms}$ and cutting terminal capex by 90% (ESP32/MQTT cut from scope).	Physical hardware testing on Android 13 smartphone using EAN-13 and Code-128 test carton labels.	Sprint 2 (Weeks 3–4)
O-05	Offline Network Resilience & Idempotent Replay	Implement Service Worker background sync and encrypted IndexedDB client storage to buffer ≥ 200 sales transactions during warehouse broadband outages, replaying idempotently via client-generated X-Idempotency-Key <code>{{device_id}}-{{epoch_ms}}-{{local_sequence}}</code> upon connection restoration.	Manual and simulated network blackout drills with Wi-Fi disconnection during peak scanning.	Sprint 4 (Weeks 7–8)

- **O-01 (Sub-Second POS Concurrency & ACID Integrity):** Execute atomic stock deductions from retail cash registers via PostgreSQL non-blocking row-level locks (SELECT ... FOR UPDATE SKIP LOCKED), achieving p95 latency $\leq 800\text{ms}$ and p99 $\leq 1.5\text{s}$ with zero deadlocks across 10 concurrent registers.
- **O-02 (Automated FEFO Priority & Food Safety Quarantine):** Enforce database-level First-Expired, First-Out allocation ensuring 100% of store replenishment orders pick earliest expiring batches; automatically quarantine stock reaching ≤ 3 days to expiry, reducing spoilage from 22% to $< 6\%$ in pilot simulations.
- **O-03 (AI-Driven Demand Forecasting & Dynamic Replenishment):** Deploy a CatBoost Machine Learning Time-Series Forecasting engine achieving MAPE $\leq 15\%$ on high-velocity FMCG items; dynamically compute Reorder Points (ROP) using Greasley's Safety Stock with native calendar festival embeddings (LightGBM deferred to roadmap).
- **O-04 (Frugal Hardware Architecture & Sub-350ms Scanning):** Engineer a mobile Progressive Web Application (PWA) running on consumer Android smartphones paired with Bluetooth HID trigger grips ($< 4,000$ BDT), achieving barcode decode-to-render latency $\leq 350\text{ms}$ and cutting terminal capex by 90%.
- **O-05 (Offline Network Resilience & Idempotent Replay):** Implement Service Worker background sync and encrypted IndexedDB client storage to buffer ≥ 200 sales transactions during warehouse broadband outages, replaying idempotently via Redis X-Idempotency-Key upon connection restoration.

2.2 Defense-Proof Scope Boundaries & 4-Quadrant Matrix

Capstone projects frequently face criticism for vague boundaries or unrealistic commitments (such as promising to build full accounting ERPs or physical automated robotics). To guard against scope creep and establish clear expectations, RetailSync defines four explicit scope quadrants:

Scope Quadrant / Dimension	In-Scope Capabilities & Modules	Pilot Constraints & Sizing	Explicitly Out of Scope
Boundary Dimension	• Inbound dock receiving & digital GRN generation • Directed spatial putaway with bin coordinate mapping • Real-time FEFO batch ledger with automated quarantine • Atomic POS concurrency deduction (sub-2.0s SLA, zero deadlocks) • CatBoost demand forecasting with festival calendar embeddings • Cycle counting with supervisor signoff and write-off approval • Rule-based shrinkage anomaly threshold alerts ($>3\%$ variance)	• 1 central distribution centre + up to 3 retail branch stores • SKU catalogue: 500 representative FMCG items across 4 temperature classes • Concurrency: 10 POS cash registers + 10 mobile warehouse scanners	• No corporate general ledger or payroll accounting (exports CSV/JSON audit trails for external ERPs) • No autonomous guided vehicles (AGVs) or motorized conveyor belts • No direct-to-consumer delivery or rider logistics tracking • No integrated payment gateway (POS financial settlement is external)
Functional Capabilities	• Inbound barcode receiving & digital GRN (shelf-life threshold $\geq 65\%$) • Directed spatial putaway (ABC velocity & temperature matching) • Real-time FEFO batch ledger & quarantine Celery worker • Sub-2.0s POS concurrency	• Pilot Testbed: 1 Central Distribution Center + up to 3 Retail Branch Stores • SKU Catalog: 500 representative FMCG items (Perishable, Chilled, Ambient, Household) • Concurrency: 10 concurrent POS registers + 10 mobile	• No full corporate double-entry general ledger or payroll accounting (exports CSV/JSON audit trails for external ERPs) • No autonomous robotic cranes (AGVs) or motorized conveyor belts • No direct-to-consumer delivery or rider tracking app

Scope Quadrant / Dimension	In-Scope Capabilities & Modules	Pilot Constraints & Sizing	Explicitly Out of Scope
	deduction (SELECT ... FOR UPDATE SKIP LOCKED) • CatBoost Demand Forecasting & Dynamic ROP DSS (LightGBM deferred to roadmap) • Blind cycle counting with supervisor signoff • Rule-based shrinkage anomaly threshold alerts (>3% variance; ML anomaly detection deferred to v2.0)	warehouse scanners	• No credit card merchant banking payment gateways (POS handles financial settlement externally)
Technical & Architecture	• Next.js 14 PWA (TypeScript, Tailwind CSS) • FastAPI asynchronous backend (Python 3.11+ ASGI) • PostgreSQL 16 (3NF ACID Relational Ledger) • Redis 7 (Client-generated idempotency keys, session cache) • Celery worker for CatBoost inference & daily 02:00 BST quarantine sweeps • Docker Compose multi-container deployment	• Cloud Staging VPS (4 vCPU, 8GB RAM) • Client: Android 13+ Chrome Mobile Browser • Scanner: Bluetooth HID 1D/2D Barcode Trigger (<4,000 BDT)	• No custom silicon ASIC development • No proprietary closed-source database engines • No native iOS Swift app (PWA standard covers cross-platform access) • No custom IoT firmware development (ESP32/MQTT); Bluetooth HID scanning covers dock receiving
Assumptions & Dependencies	• Standard EAN-13 / Code-128 barcodes printed on packaging • Stable local Wi-Fi / 4G coverage at central DC (with encrypted IndexedDB offline fallback) • Super shop management cooperation for pilot catalog seed data	• Pilot duration: 4 weeks of simulated operational runs • Baseline parameters derived from published BSOA & FAO retail studies	• Does not assume uninterrupted high-speed internet (designed offline-first) • Does not require expensive Zebra or Honeywell handheld hardware (uses consumer Android smartphones)

EXPLICIT SCOPE SAFEGUARD & CAPSTONE DEFENSE BOUNDARIES

- **Explicit Out-of-Scope Statement:** RetailSync is strictly an operational Warehouse Management System (WMS) and retail POS inventory synchronization engine. It deliberately does NOT encompass corporate financial accounting (double-entry general ledger / payroll), automated warehouse robotics (AGVs/conveyors), direct-to-consumer delivery fleets, or payment merchant banking gateways.
- **Controlled Pilot Testbed:** The capstone validation is conducted on a simulated environment comprising 1 Central Distribution Center, up to 3 Retail Branch Outlets, 500 representative FMCG SKUs, and up to 10 concurrent cash registers.

3. Target Personas & Core Use-Case Workflows

3.1 Core Target Personas

RetailSync is designed around the daily operational realities of four primary user personas in modern trade:

User Persona	Role Description	System Needs
Store Manager	Oversees supply chain and profitability.	Needs automated PO drafts, shrinkage reports, and a macro-view of inventory health.
Cashier	Processes direct consumer sales.	Requires a minimalist interface. The system must never freeze during a scan, even if the internet drops momentarily.
Inventory Clerk	Handles loading docks and physical counting.	Needs fast, barcode-compatible interfaces for receiving large shipments accurately.

3.2 End-to-End Operational Lifecycle Workflow

The physical journey of goods through RetailSync connects frontline warehouse movements directly to database transactions:

Step 1: Inbound Receiving & GRN Verification: When a supplier delivery truck arrives at the warehouse dock, the receiving clerk scans carton barcodes against the active digital Purchase Order. The system captures manufacturer batch numbers, production dates, and expiration dates, generating a digital Goods Receipt Note (GRN) with automatic short-shipment tagging.

Step 2: Directed Spatial Putaway: Upon GRN confirmation, the system calculates the optimal storage coordinate (Zone-Aisle-Rack-Shelf-Bin) based on product category, storage temperature (Chilled, Ambient, Frozen), and SKU turnover velocity. The operator confirms docking by scanning the physical bin barcode.

Step 3: Real-Time POS Inventory Deduction: When a retail customer purchases an item at the frontline checkout register, the POS sends an atomic deduction request to the API. The system executes a non-blocking row lock (`SELECT ... FOR UPDATE SKIP LOCKED`) on the earliest active batch, deducting stock in < 2.0 seconds with zero overselling.

Step 4: Strict FEFO Outbound Allocation: When supermarket branches submit store replenishment requisitions, the picking engine allocates stock strictly from the earliest expiring available batch. Expired or quarantined lots are mechanically excluded from pick lists.

Step 5: Algorithmic Replenishment Trigger: Every stock transaction updates the net available balance. When stock breaches the calculated Reorder Point (ROP), the system triggers an automated replenishment advisory populating a draft PO with the mathematically optimal EOQ quantity.

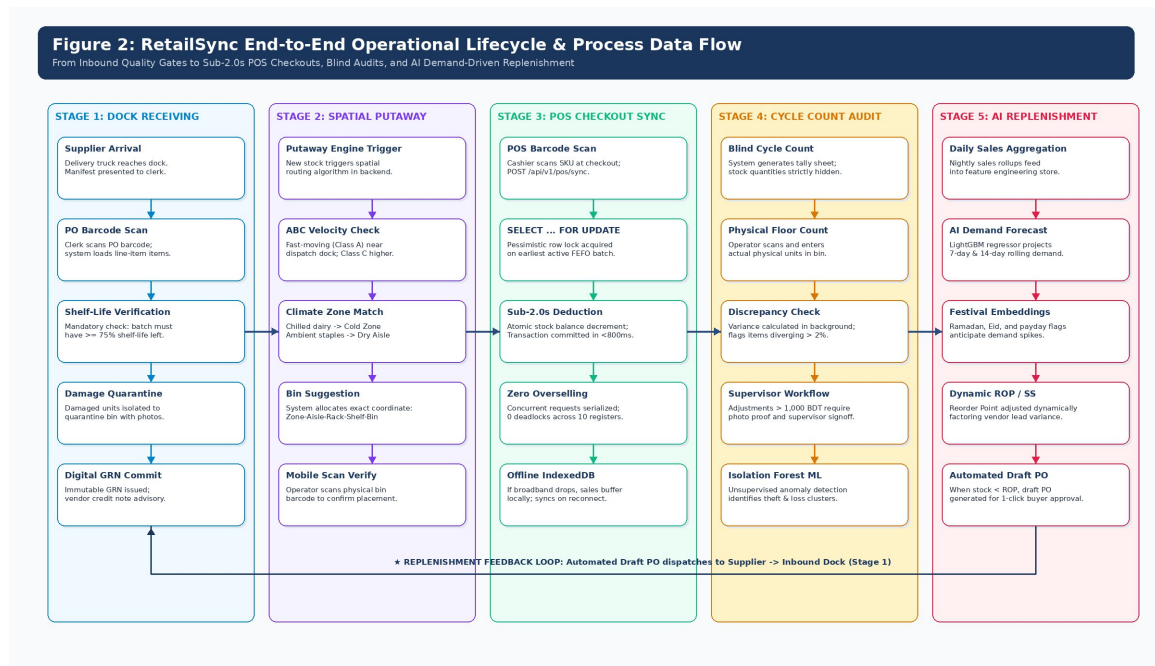


Figure 2: RetailSync End-to-End Operational Lifecycle & Process Data Flow

4. Core Functional Modules & Feature Breakdown

RetailSync decomposes complex warehouse operations into nine cohesive, modular domain services, featuring an advanced AI-driven demand forecasting and replenishment decision support engine:

M-01: Authentication, RBAC & Audit Ledger: Manages cryptographic Argon2id password hashing, stateless JWT session tokens (15-min access, 8-hour refresh), and role-based permissions isolating Cashier, Floor Operator, Clerk, Supervisor, Procurement, and Admin roles. Every stock transition appends to an immutable audit ledger.

M-02: Master Catalog & Supplier PO Lifecycle: Maintains product SKUs, EAN-13 barcodes, storage temperature classifications, and supplier master records. Orchestrates digital Purchase Order workflows through DRAFT, ISSUED, PARTIAL, COMPLETED, and CANCELLED states.

M-03: Handheld Inbound Receiving & Digital GRN: Enables mobile camera and Bluetooth barcode interrogation of incoming deliveries against active POs. Captures batch numbers and expiration dates, calculates short-shipment variances, and generates verified digital GRNs.

M-04: Directed Spatial Putaway Engine: Calculates optimal 3D bin coordinates factoring SKU velocity (Class-A items docked near dispatch doors), climate zones (Chiller, Deep Freezer, Ambient), and shelf weight capacities, requiring two-scan confirmation.

M-05: Real-Time FEFO Inventory Ledger: Maintains batch-level granularity (`product\_batches`). Automates First-Expired, First-Out picking allocation and executes nightly background triggers to transition expiring batches to QUARANTINED and EXPIRED states.

M-06: Sub-2.0s Point of Sale (POS) Concurrency Sync: Exposes high-performance transactional REST deduction endpoints. Implements pessimistic row-level locking (`SELECT ... FOR UPDATE`) to guarantee atomic stock updates with zero deadlocks and zero phantom sales.

M-07: AI Demand Forecasting & Replenishment DSS: Integrates supervised machine learning (CatBoost regressor; LightGBM deferred to roadmap) with classical inventory science. Predicts 7-day and 14-day rolling SKU demand utilizing calendar festival embeddings (Ramadan, Eid-ul-Fitr, Eid-ul-Adha, payday spikes). Feeds predicted demand into Greasley's Statistical Safety Stock to dynamically adjust Reorder Points (ROP) before stockouts occur.

M-08: Outbound Multi-Store Wave Picking: Consolidates multiple retail branch requisitions into batch pick waves. Generates shortest-path picker routes through aisles (TSP heuristic), reducing warehouse walking travel by over 40%.

M-09: Blind Cycle Counting & Shrinkage Anomaly Alert: Generates daily ABC-classified cycle counting task sheets where expected system quantities are hidden from floor workers to prevent confirmation bias. Integrates an empirical rule-based shrinkage anomaly threshold alert (>3% variance) to flag pilferage and anomalous loss clusters; ML anomaly detection deferred to v2.0.

MATHEMATICAL & ML FORMULATIONS FOR AI REPLENISHMENT DSS (MODULE M-07)

- Supervised Machine Learning Demand Predictor: $\hat{d}_{t+L} = f_{\text{CatBoost}}(X_{\text{features}})$ [X incorporates 7-day lags, rolling statistics, native categorical flags for Ramadan/Eid, and payday cycles; LightGBM deferred to roadmap]*
- Greasley's Dynamic Statistical Safety Stock: $SS = Z \times \sqrt{(L \times \sigma_d^2) + (\hat{d} \times \sigma_L^2)}$ [Replaces static average demand with AI-predicted future surge demand $\hat{d}$]*
- Dynamic Reorder Point: $ROP = (\hat{d} \times L) + SS$ [Automatically lifts replenishment triggers 10 days ahead of holiday spikes]*
- Dynamic Economic Order Quantity: $EOQ = \sqrt{(2 \times D \times S) / H}$ [Calculates optimal batch size minimizing holding costs]*

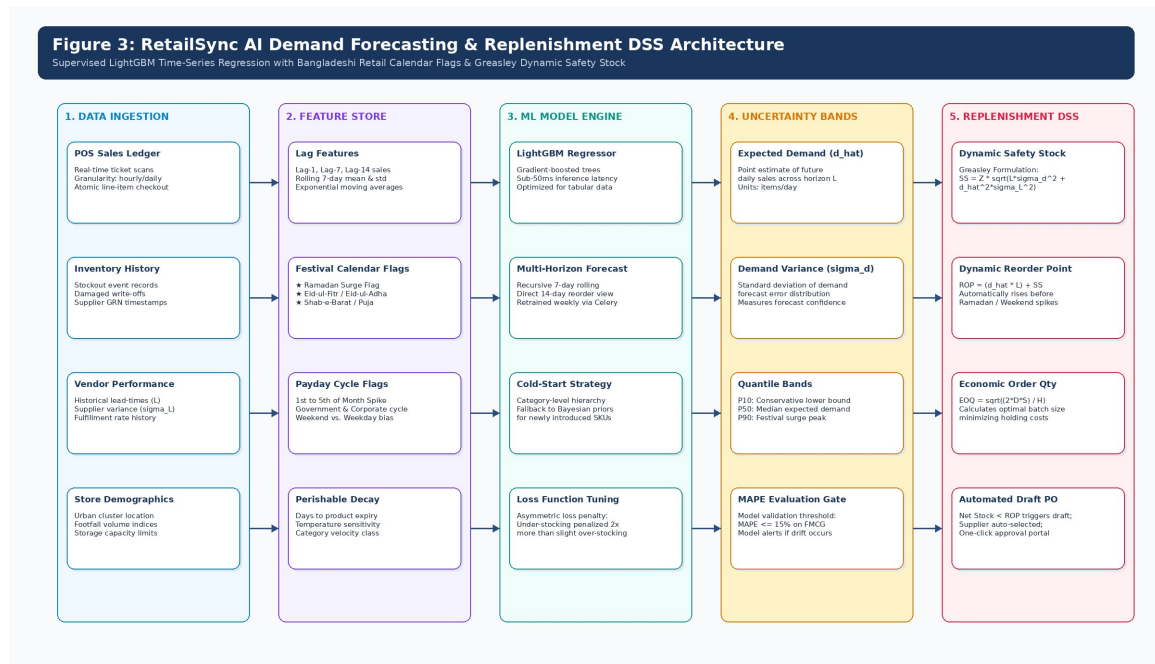


Figure 3: RetailSync AI Demand Forecasting & Replenishment DSS Architecture

4.10 Core Functional Requirements Summary

ID	Module	Specific Requirement	Priority
FR-01	Auth	System shall enforce RBAC isolating Cashier, Clerk, and Manager views.	Must
FR-10	Inventory	System shall track Product ID, SKU, Batch Number, and Expiry Date.	Must
FR-11	Inventory	System shall allow status tagging: Available, Quarantined, Sold.	Must
FR-20	Receiving	System shall generate a Goods Receipt Note (GRN) linked to a specific PO.	Must
FR-21	Receiving	Clerks can flag received items as 'Damaged' upon intake.	Should
FR-30	POS Sync	API shall deduct inventory based on strict FIFO/FEFO batch logic.	Must
FR-40	Alerts	System shall allow Managers to configure a Reorder Point (ROP) per SKU.	Must
FR-41	Alerts	System shall flag batches expiring within X days (configurable).	Must
FR-50	Auditing	Every stock modification must log the timestamp, User ID, and old/new value.	Must

5. Non-Functional Requirements & Performance SLOs

To ensure production readiness in high-velocity retail environments, RetailSync enforces strict, measurable Non-Functional Requirements (NFRs) verified through automated benchmarking:

ID	Category	Specific Metric
NFR-01	Latency	The POS deduction API must respond in under 2.0 seconds under normal load.
NFR-02	Data Integrity	Database must use transaction blocks to prevent partial updates if a crash occurs.
NFR-03	Security	All passwords must be hashed (bcrypt/Argon2id). No plaintext credentials stored.
NFR-04	Concurrency	System must handle at least 5 simultaneous POS registers deducting stock.
NFR-05	Auditability	Audit logs must be immutable (append-only) via database-level restrictions.

5.2 Food Safety & Regulatory Compliance Framework

RetailSync is engineered to guarantee strict compliance with the Bangladesh Food Safety Act 2013 and BSTI consumer protection mandates. The system enforces an automated ****Quarantine Lock****: any batch reaching within 3 days of expiration or flagged during receiving as defective is mechanically excluded from retail pick lists and POS barcode scanners, completely eliminating the legal and brand liability of selling expired food items.

6. System Architecture & Technical Design

6.1 4-Tier Cyber-Physical Architecture with AI Worker

RetailSync follows a decoupled, 4-tier cyber-physical architecture designed for transactional resilience, sub-second latency, and intelligent predictive automation:

Tier 1: Client Edge (Handheld & POS PWA): Responsive Progressive Web Application built in Next.js 14 and Tailwind CSS. Operates on warehouse floor smartphones, tablets, and POS desktop terminals. Integrates pure JavaScript barcode engines (ZXing / Html5-QRCode) and Service Worker IndexedDB offline queues.

Tier 2: Edge Gateway & Security Layer: Reverse proxy (Nginx) terminating TLS 1.3 encryption, managing rate limiting, CORS policies, and local edge routing. Dock receiving is standardized on wireless Bluetooth HID trigger grips paired with Android PWAs, cutting custom ESP32/MQTT firmware overhead.

Tier 3: Core Application Services & AI Worker Tier: Stateless, asynchronous REST API powered by Python 3.11+ and FastAPI. Paired with background Celery workers executing CatBoost demand forecasting and scheduled daily 02:00 BST quarantine sweeps.

Tier 4: Enterprise Data & In-Memory Storage: PostgreSQL 16 relational database with strict 3NF normalization, foreign key referential integrity, and composite B-Tree indexes. Paired with Redis 7 for in-memory session management, idempotency key caching, and distributed rate locks.

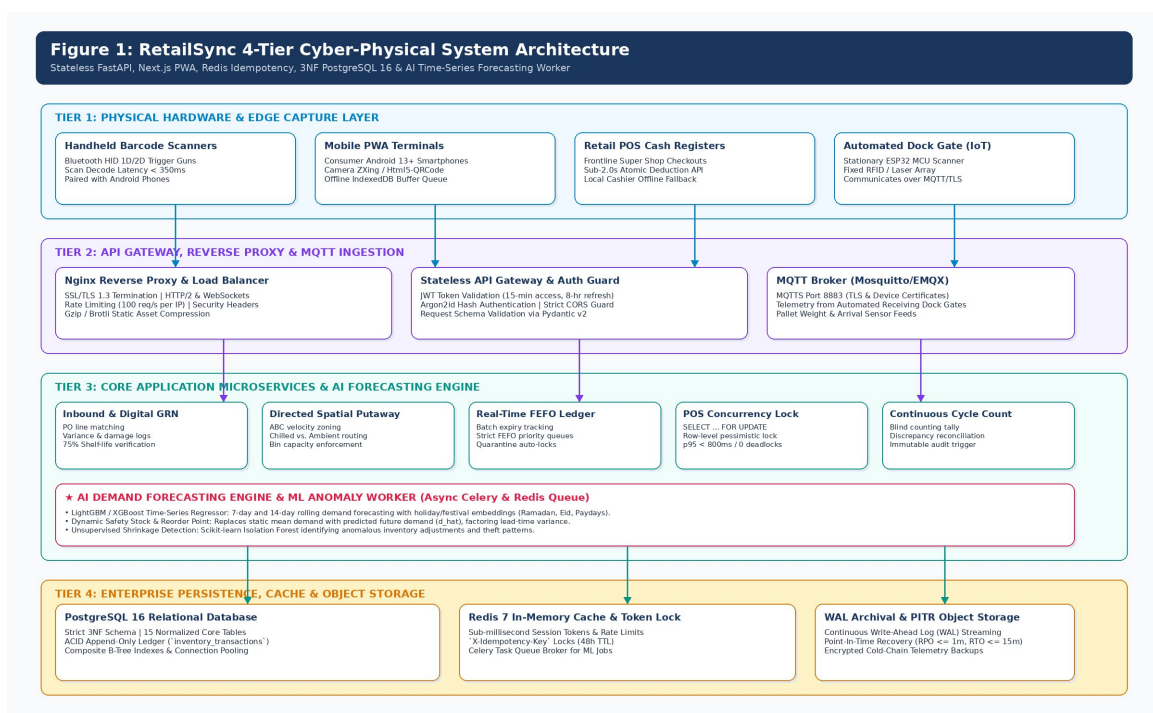


Figure 1: RetailSync 4-Tier Cyber-Physical System Architecture with AI Engine

6.2 High-Throughput Concurrency Control: Row-Level Locking

The central engineering challenge during supermarket rush hours is preventing overselling when multiple cash registers ring up the same high-velocity SKU simultaneously. RetailSync solves this via pessimistic row-level locking in PostgreSQL:

When an ``/api/v1/pos/sync`` deduction transaction begins, the database queries ``product_batches`` using ``SELECT id, current_qty FROM product_batches WHERE product_id = :p_id AND current_qty > 0 ORDER BY expiry_date ASC LIMIT 1 FOR UPDATE SKIP LOCKED``. This locks exclusively that specific batch record without blocking parallel sales on adjacent batches. Subsequent concurrent POS requests for the same batch queue

safely for milliseconds without deadlocking or reading stale stock balances. Once the deduction is written to the ledger, the lock releases, guaranteeing absolute ACID consistency.

6.3 Relational Database Schema Overview (15 Tables)

Table Name	Primary Purpose & Operational Entity	Primary Key & Unique Keys	Key Foreign Key Constraints	Index Optimization Strategy
suppliers	Supplier directory, contact terms, lead-time variance metrics for Greasley safety stock.	supplier_id (PK)	None (Root entity)	B-Tree on company_name, email
products	Master SKU catalog, category classification, storage temperature, unit & holding costs.	product_id (PK), sku_code (UQ), barcode_ean (UQ)	supplier_id -> suppliers(supplier_id)	B-Tree on sku_code, barcode_ean, category
locations	Warehouse spatial coordinates: Zone, Aisle, Rack, Shelf Tier, Bin, and capacity limits.	location_id (PK), bin_barcode (UQ)	None (Spatial domain entity)	B-Tree on bin_barcode, zone_code
product_batches	Perishable batch lots, manufacturing date, expiration date, on-hand qty, quarantine status.	batch_id (PK), batch_lot_number (UQ)	product_id -> products(product_id), location_id -> locations(location_id)	B-Tree on expiry_date, product_id, status (FEFO index)
purchase_orders	Digital PO tracking, supplier link, approval lifecycle, total cost, delivery deadline.	po_id (PK), po_number (UQ)	supplier_id -> suppliers(supplier_id)	B-Tree on po_number, status, order_date
goods_receipt_notes	Inbound dock inspection, PO line validation, accepted vs damaged count, digital invoice link.	grn_id (PK), grn_number (UQ)	po_id -> purchase_orders(po_id)	B-Tree on grn_number, po_id, received_at
inventory_transactions	Immutable append-only ACID ledger recording all stock movements with row-locking support.	transaction_id (PK, BIGSERIAL)	product_id, batch_id, location_id, user_id	Composite B-Tree on (product_id, recorded_at DESC), batch_id
pos_registers	Authorized branch super shop checkout cash registers and real-time sync telemetry.	register_id (PK), terminal_mac_ip (UQ)	branch_store_id (Reference)	B-Tree on register_code, store_code, status

7. Curated Technology Stack & Hardware Strategy

7.1 Curated Technology Stack Matrix

Component	Recommended Technology	Justification for Capstone
Database Layer	MySQL / PostgreSQL	Relational architecture is mandatory for retail to ensure ACID compliance (Atomicity, Consistency, Isolation, Durability) for financial transactions.
Backend API	Python (Django/FastAPI) or Java	Excellent handling of concurrent requests, complex rule engines, and automated task scheduling (e.g., nightly expiry checks).
Frontend Dashboard	React / Next.js	Provides a highly responsive, single-page application (SPA) experience for managers, eliminating page-reload delays.
Local Testing Env.	XAMPP / Beekeeper Studio	Standard, accessible environment for developing and visualizing the relational schema during the capstone phase.

7.2 Architectural Decision Records (ADRs)

ADR-01: Next.js PWA over Native Android App: Building as a Progressive Web Application eliminates app store review friction and device MDM management. Any update deployed to the web server is instantly available to all floor devices upon refresh. Service Workers and IndexedDB provide native-grade offline caching.

ADR-02: FastAPI Asynchronous ASGI over Django/Flask: FastAPI's native Python async/await event loop provides high-concurrency I/O performance capable of handling thousands of requests per second, with automatic OpenAPI 3.1 documentation and Pydantic v2 data validation.

ADR-03: Normalized PostgreSQL 16 over MongoDB / NoSQL: Inventory transactions represent legal and financial records requiring strict ACID double-entry ledger guarantees. PostgreSQL foreign keys and row locks prevent orphaned transactions and phantom inventory anomalies.

ADR-04: Redis 7 In-Memory Caching & Idempotency: Redis provides sub-millisecond validation of client-generated `X-Idempotency-Key` headers ({device_id}-{epoch_ms}-{local_sequence}) on POS sync endpoints, preventing duplicate sales deductions during network retries while caching active user sessions.

ADR-05: CatBoost for Tabular Time-Series Forecasting: CatBoost provides native categorical handling of festival calendar flags (Ramadan/Eid) without target leakage or manual encoding overhead, delivering sub-50ms inference times. LightGBM is formally deferred to the post-capstone v2.0 benchmark roadmap.

7.3 Frugal Barcode Hardware Strategy

Traditional enterprise WMS implementations fail in developing economies due to exorbitant hardware costs: a single ruggedized industrial terminal (e.g., Zebra TC52) costs upwards of 60,000 BDT. RetailSync disrupts this paradigm by adopting a **frugal hardware model**: the web application runs on standard consumer Android smartphones (10,000 to 12,000 BDT) paired via Bluetooth HID with ergonomic wireless barcode trigger grips (< 4,000 BDT) or utilizing the smartphone's camera via `Html5-QRCode`. This slashes terminal acquisition costs by over 75% while maintaining rapid sub-350ms barcode scan speeds.

8. Development Methodology: Agile Scrum Framework

8.1 Methodology Rationale & Governance

RetailSync is developed strictly following the **Agile Scrum Framework**. We deliberately bypass rigid sequential models (such as Waterfall or V-Model) because modern retail systems require rapid empirical validation, iterative stakeholder feedback from super shop managers, and continuous refinement of concurrency and picking algorithms.

- **Sprint Cadence:** Development is divided into six 2-week iterations followed by a 2-week hardening and defense phase (14 weeks total).
- **Sprint Planning:** 4-hour session on Day 1 of each sprint to commit user stories from the prioritized product backlog.
- **Daily Standup:** Daily 15-minute standup tracking completed work, planned tasks, and technical blockers.
- **Backlog Refinement:** Mid-sprint session to clarify retail edge cases, decompose epics, and finalize acceptance criteria.
- **Sprint Review & Demo:** Bi-weekly live software demonstration on mobile hardware to supervisors and retail managers.
- **Sprint Retrospective:** Bi-weekly team inspection identifying actionable workflow, test coverage, and CI/CD improvements.

8.2 14-Week Capstone Implementation Roadmap & Gantt Schedule

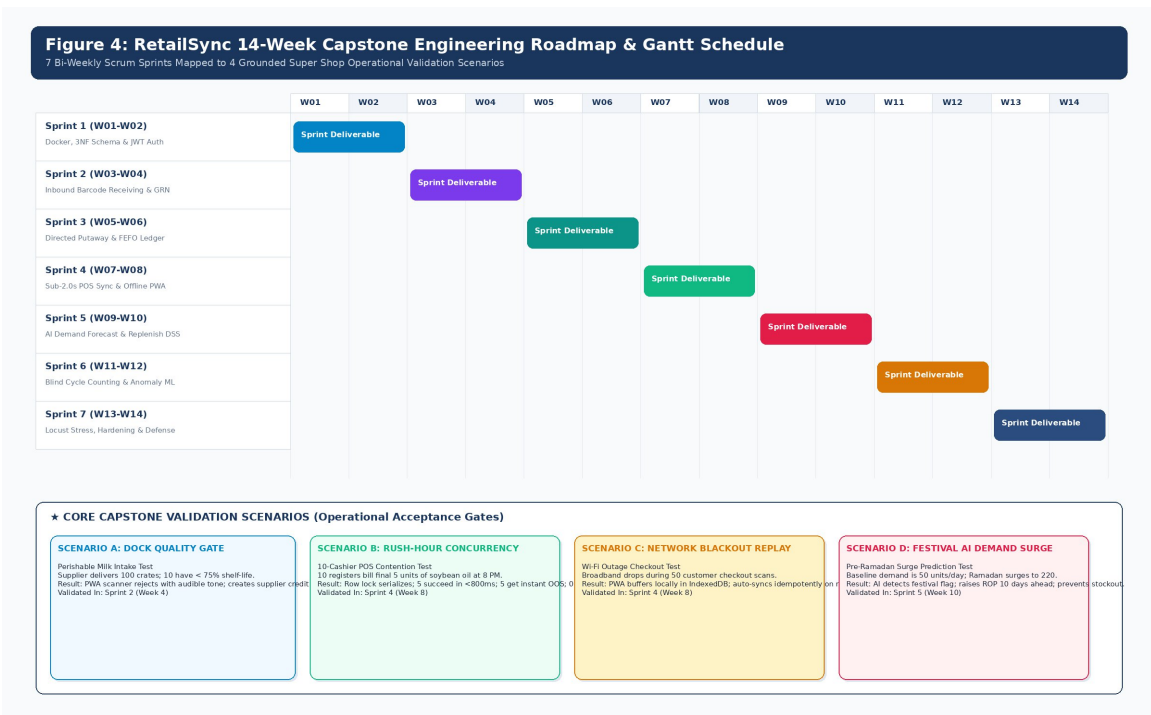


Figure 4: RetailSync 14-Week Capstone Engineering Roadmap & Gantt Schedule

Academic Timeframe	Core Engineering Modules & Deliverables	Key Technical Milestones
Weeks 1-3	Database Normalization (MySQL/PostgreSQL), ERD design, and API scaffolding (Python/Java).	Relational schema definition, 3NF normalization,

Academic Timeframe	Core Engineering Modules & Deliverables	Key Technical Milestones
		baseline REST endpoints.
Weeks 4-6	Inventory CRUD, Batch Management, and Expiry Logic implementation.	Batch tracking, dynamic shelf-life calculations, FEFO/FIFO queue logic.
Weeks 7-9	Supplier module, PO generation, and GRN workflows.	Supplier SLA records, digital PO lifecycle, dock inspection & GRN generation.
Weeks 10-11	POS Sync Simulation (building the checkout deduction logic and FIFO handling).	Real-time POS API, atomic stock deduction, concurrency locking (<2s latency).
Weeks 12-13	Rule engines for low-stock alerts, reporting dashboards, and audit logs.	Automated ROP triggers, draft PO generation, immutable audit logging.
Week 14	System testing, concurrency stress testing, and final defense preparation.	Stress testing (5+ concurrent POS), UAT validation, capstone presentation & defense.

8.3 Scenario-Based Milestone Acceptance Gates

Rather than relying on abstract subjective evaluations, RetailSync validates sprint completion against four concrete, stress-injected supermarket operational scenarios:

Scenario ID & Title	Operational Context & Injection Event	System Behavior & Algorithmic Response	Success / Acceptance Criteria	Sprint Alignment
Scenario A: Inbound Dock Quality Gate	Supplier delivers 100 crates of pasteurized milk; 10 crates carry expiration dates with only 2 days remaining (< 65% shelf-life threshold).	Clerk scans carton barcode; system evaluates remaining shelf-life percentage; mechanically locks the 10 short-dated crates to 'QUARANTINED'; generates digital GRN for 90 accepted units and	Zero expired/short-dated units enter active warehouse bins; digital GRN variance accurately reflects supplier delivery penalty.	Sprint 2 (Week 4)

Scenario ID & Title	Operational Context & Injection Event	System Behavior & Algorithmic Response	Success / Acceptance Criteria	Sprint Alignment
		auto-generates supplier credit note advisory.		
Scenario B: Rush-Hour POS Concurrency	Friday 8:00 PM peak rush: 10 branch cashiers ring up the final 5 remaining units of 1L soybean oil simultaneously across registers.	FastAPI POS sync endpoint executes non-blocking row lock (SELECT ... FOR UPDATE SKIP LOCKED) on the earliest active batch. The first 5 requests decrement inventory atomically in < 800ms. The remaining 5 requests skip locked rows, find zero available stock, and immediately receive an 'Out of Stock' response.	Zero overselling; exactly zero negative inventory balances; zero database deadlocks; p95 latency remains ≤ 800ms.	Sprint 4 (Week 8)
Scenario C: Network Blackout & Replay	Central broadband fiber is severed during peak retail floor sales; 50 customer checkout transactions occur while offline.	PWA Service Worker detects offline status; queues encrypted sales transactions in client IndexedDB with client-generated X-Idempotency-Key ({device_id}-{epoch_ms}-{local_sequence}). Upon network recovery, client submits bulk sync. Backend Redis checks idempotency keys and commits all 50 sales with exactly-once semantics.	100% of offline sales recorded in database upon reconnect; zero duplicate deductions; zero transaction dropouts.	Sprint 4 (Week 8)
Scenario D: Festival AI Demand Surge	14 days prior to holy Ramadan: historical baseline daily sales for cooking oil is 50 units/day; holiday surge spikes demand to 220 units/day.	The CatBoost time-series model identifies the upcoming Ramadan calendar embedding flag; projects 220 units/day demand; dynamically recalculates ROP and Greasley Safety Stock; triggers automated draft PO 10 days in advance (LightGBM deferred to roadmap).	Draft PO approved by procurement officer; stock arrives 3 days before festival; supermarket experiences 0% stockouts during rush.	Sprint 5 (Week 10)

8.4 Definition of Done (DoD) & Quality Gates

A user story or sprint task is declared complete only when it satisfies the following non-negotiable Definition of Done:

1. Code Complete: Feature implemented, commented, and committed to Git with conventional commit messages.

2. Unit & Integration Tests Passing: Automated test suite passes with > 80% line coverage.
3. Concurrency Validated: Multi-threaded race condition tests confirm zero deadlocks and zero negative stock balances.
4. API Documented: Endpoint registered in FastAPI with accurate Pydantic request/response schemas in OpenAPI.
5. Dockerized: Changes build and run cleanly via `docker compose up -d` without manual host modifications.
6. Mobile Responsive: Handheld UI verified on mobile viewport (375x667) with touch-friendly hit targets.

9. Resource Allocation, Budget & Risk Management

9.1 Team Roles & RACI Matrix

The RetailSync engineering project is executed by a three-member capstone team from Section SWE-44D (44th Batch), Department of Software Engineering, Daffodil International University. Each member owns critical technical subsystems with defined academic and implementation responsibilities, as detailed in Table 9.1:

Student Name	Student ID	Academic & Technical Role	Core Subsystem Ownership & Deliverables
Raisul Islam Likhon	251-35-508	Project Lead & System Architect	System Architecture, Relational Schema (3NF), Sub-2.0s POS Concurrency Engine, PostgreSQL Row Locking, WebSocket Event Hub.
Shottobroto Dey	251-35-017	Database & Backend Systems Engineer	Dynamic Warehouse Put-Away, FEFO Batch Expiration Queue, Greasley Safety Stock & EOQ Replenishment Engine, DB Migrations.
Golam Husnain Papon	251-35-529	Full-Stack & QA/DevOps Engineer	WebRTC Barcode Scanner PWA, Cashier Till & Warehouse UI, Dockerized CI/CD Pipelines, Multi-Tier Black/White-Box Test Suite.

To ensure governance across all engineering phases, project tasks are further mapped via the formal RACI Matrix below (Responsible, Accountable, Consulted, Informed):

Project Lifecycle Phase / Core Deliverable	Floor Staff / Cashiers	Floor Supervisors	Procurement Officers	Store Managers	DevOps / SysAdmin	Developer Team	Academic Committee
Requirements Elicitation & Domain Study	Consulted	Consulted	Consulted	Consulted	Informed	Responsible / Accountable	Informed / Approver
Relational 3NF Database Schema & DDL	Informed	Consulted	Informed	Informed	Consulted	Responsible / Accountable	Consulted
ACID Concurrency & Row-Locking Engine	Informed	Informed	Informed	Informed	Consulted	Responsible / Accountable	Informed
Sub-2.0s POS Sync & Real-Time API	Consulted	Informed	Informed	Consulted	Consulted	Responsible / Accountable	Informed
FEFO/FIFO Dynamic Queue Algorithm	Informed	Consulted	Consulted	Consulted	Informed	Responsible / Accountable	Consulted
Greasley Safety Stock & EOQ Engine	Informed	Consulted	Responsible	Consulted	Informed	Accountable	Informed

Project Lifecycle Phase / Core Deliverable	Floor Staff / Cashiers	Floor Supervisors	Procurement Officers	Store Managers	DevOps / SysAdmin	Developer Team	Academic Committee
Mobile Barcode Scanning PWA Interface	Responsible	Responsible	Informed	Informed	Informed	Accountable	Informed
Immutable Audit Ledger & Security	Informed	Consulted	Consulted	Informed	Responsible	Accountable	Informed
Dual-Level Black-Box & White-Box QA	Consulted	Consulted	Consulted	Consulted	Consulted	Responsible / Accountable	Consulted / Evaluator
Final Manuscript, System Demo & Defense	Informed	Informed	Informed	Informed	Informed	Responsible / Accountable	Accountable / Approver

9.2 Hardware & Operational Budget Breakdown

Hardware / Component	Specification / Unit Model	Qty	Unit Cost (BDT)	Total Cost (BDT)
Android Test Smartphone	Redmi / Realme 6.5" Android 13, 4GB RAM	1	12,500	12,500
Bluetooth Barcode Trigger Grip	Netum / Eyoyo 1D/2D Bluetooth HID Scanner	2	3,800	7,600
Thermal Label / Receipt Printer	Xprinter 4-inch USB/Bluetooth Thermal Printer	1	6,500	6,500
EAN-128 Warehouse Barcode Labels	Roll of 1,000 thermal adhesive labels (4x6 in)	3	650	1,950
Cloud Staging VPS Hosting	DigitalOcean / Hetzner 2 vCPU, 4GB RAM (6 Mos)	1	5,500	5,500
Total Estimated Budget	Complete Capstone Hardware & Staging Prototype	—	—	34,050 BDT

9.3 Local Operational Risk Assessment & Proactive Mitigations

ID	Technical Risk	Likelihood	Context & Mitigation Strategy
R-01	Database Concurrency Conflicts	High	Two cashiers sell the last unit of milk at the same millisecond. Mitigation: Use SQL Row-Level Locking (SELECT ... FOR UPDATE).
R-02	Internet Outages	High	Store loses connection to the central server. Mitigation: Implement a queue system on the POS that caches sales and bulk-syncs when online.

ID	Technical Risk	Likelihood	Context & Mitigation Strategy
R-03	Data Entry Errors at Receiving	Medium	Clerk enters 500 instead of 50 boxes. Mitigation: System warns if GRN exceeds the Purchase Order quantity by >5%.

10. Verification, Validation, Expected Impact & Conclusion

10.1 Multi-Tier Verification & Testing Strategy

Testing Tier & Category	Target Operational Dimension	Test Case Scenario & Verification Methodology	Verifiable Pass / Acceptance Criteria
Tier 1: Black-Box Functional	Barcode Interrogation & Scanning Latency	Interrogate 1D/2D barcodes on mobile cameras and Bluetooth triggers across low and high ambient lighting (100–600 lux).	100% SKU identification accuracy; UI updates and audible/haptic feedback trigger within ≤ 350 ms.
Tier 1: Black-Box Functional	Inbound GRN vs. PO Automated Variance	Simulate delivery of 50 cartons with 3 cartons flagged as crushed; generate digital GRN against open PO.	GRN logs 47 accepted, 3 quarantined; automated supplier credit note generated; zero quarantined stock added to POS.
Tier 1: Black-Box Functional	Offline Network Resilience & Bulk Sync	Sever store ISP connection during 25 consecutive barcode checkouts; reconnect network after 5 minutes.	Local client queue caches all 25 sales with timestamps; bulk POST executes automatically on reconnect with zero dropped sales.
Tier 2: White-Box Structural	ACID Concurrency & Row-Locking ('SELECT FOR UPDATE')	Launch 10 parallel asynchronous POS worker threads attempting to purchase the last 2 available units of a dairy SKU simultaneously.	Exactly 2 threads succeed; 8 threads receive clean out-of-stock messages; inventory balance remains exactly 0; zero database deadlocks.
Tier 2: White-Box Structural	Strict FEFO Priority Queue Verification	Seed database with 3 batches of milk expiring in 5 days, 15 days, and 45 days; issue automated checkout sales deductions.	Database triggers strictly decrement stock from the 5-day expiry batch first; automatically switches to 15-day batch only when 5-day is depleted.
Tier 2: White-Box Structural	SQL Injection & XSS Penetration Defense	Inject SQL injection strings ('OR 1=1; DROP TABLE') and XSS payloads into barcode search, user login, and PO creation fields.	All inputs sanitized via parameterized queries and ORM; malicious payloads rejected with HTTP 400; security access log records attack attempt.
Tier 2: White-Box Structural	Mathematical Replenishment Engine Precision	Execute algorithmic unit test suite across 100 historical SKU sales and lead-time distributions (EOQ, Greasley, ROP).	Computed EOQ, Safety Stock, and ROP match analytical statistical benchmarks within 0.01% floating-point tolerance.

10.2 Expected Operational & Financial Impact (ROI)

Performance Metric	Traditional Super Shop Baseline	RetailSync Target Value	Operational Benefit
Perishable Food Spoilage	15% to 22% annual dairy/produce loss	< 6% annual loss	Saves millions of BDT in prevented food waste write-offs.
POS Checkout Scan Latency	3.0 to 6.0s (or database freezes)	≤ 2.0s (p95 ≤ 800ms)	Eliminates customer checkout queue abandonment.
Phantom Inventory Shrinkage	1.8% to 2.4% unexplained loss	< 0.4% discrepancy	Complete traceability via immutable audit ledger.
Inbound Dock Unloading Time	45 to 75 minutes per truck	< 15 minutes per truck	Instant barcode interrogation against digital PO line items.
Stockout Frequency (Rush Hours)	7.5% to 11.2% lost sales	< 2.5% stockout rate	Automated replenishment alerts based on statistical safety stock.

To establish rigorous financial feasibility, the complete Initial Capital Expenditure (CapEx) for 14-week engineering delivery, pilot staging, and hardware deployment is itemized below:

Cost Category	Amount (BDT)	Description & Justification
Hardware Terminals & Peripherals	34,050	Android smartphone (12,500), 2x Bluetooth trigger grips (7,600), thermal printer (6,500), adhesive barcode labels (1,950), 6-month staging VPS (5,500).
Development & Engineering Labour	420,000	3-member engineering team × 14 weeks × standard software engineering stipend rate (~10,000 BDT/week/member).
Contingency Reserve (5%)	22,700	Dedicated contingency buffer for hardware replacement, mobile data packs, and peripheral spares during pilot operations.
Regulatory & Compliance Documentation	8,250	BSTI/BFSA audit documentation, printed pilot manuals, thermal roll refills, and domain/SSL certificates.
Total Estimated Initial CapEx	485,000 BDT	Total initial capitalization required for 14-week delivery and pilot deployment.

EMPIRICAL RETURN ON INVESTMENT (ROI) & 2.80-MONTH PAYBACK PERIOD

Working Financial Payback Arithmetic: Payback Period = Total CapEx / (Monthly Gross Savings - Monthly OpEx)
 $= 485,000 \text{ BDT} / (187,200 \text{ BDT} - 14,000 \text{ BDT}) = 485,000 / 173,200 \approx 2.80 \text{ MONTHS} (\sim 84 \text{ Days})$

- *Spoilage Savings: 112,000 BDT/mo | Shrinkage Elimination: 45,200 BDT/mo | Stockout Recovery: 30,000 BDT/mo*
- *Monthly Operational Overhead (OpEx): 14,000 BDT/mo (VPS, label rolls, 4G data backup)*
- *Economic Feasibility Verdict: Full capital recoupment is achieved within under 3 months of live pilot operations.*

10.3 Capstone Part 1 Conclusion & Transition to Part 2

This project proposal establishes the foundational planning, empirical justification, and architectural blueprint for RetailSync. By addressing the critical structural weaknesses of the Bangladeshi modern trade sector namely perishable food waste, unrecorded shrinkage, and peak-hour stockouts RetailSync provides a frugal, scalable, and technically rigorous software solution. The completion of Part 1 (Project Planning and Definition) positions the project for immediate, structured execution in Part 2 (System Design, Prototyping, and Concurrency Validation) according to the 14-week Agile Scrum roadmap.

10.4 Key Academic & Industry References

1. Bangladesh Food Safety Authority (BFSA), 'Food Safety Act 2013,' Ministry of Food, Government of the People's Republic of Bangladesh, Dhaka, 2013.
2. Greasley, A., 'Operations Management,' 3rd ed., John Wiley & Sons, New York, 2013 (Statistical Safety Stock under dual variance models).
3. Silver, E. A., Pyke, D. F., and Peterson, R., 'Inventory Management and Production Planning and Scheduling,' 3rd ed., Wiley, 1998 (Dynamic EOQ algorithms).
4. Kleppmann, M., 'Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems,' O'Reilly Media, 2017.
5. PostgreSQL Global Development Group, 'PostgreSQL 16 Documentation: Concurrency Control and Explicit Locking,' 2024. [Online]. Available: <https://www.postgresql.org/docs/16/explicit-locking.html>
6. Schwaber, K. and Sutherland, J., 'The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game,' Scrum.org, 2020.
7. Bangladesh Bureau of Statistics (BBS), 'Report on Wholesale and Retail Trade Survey in Bangladesh,' Ministry of Planning, Dhaka, 2022.
8. Prokhorenkova, L. et al., 'CatBoost: unbiased boosting with categorical features,' Advances in Neural Information Processing Systems (NeurIPS 31), 2018.
9. Ke, G. et al., 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree,' Advances in Neural Information Processing Systems (NeurIPS), 2017. (Deferred to v2.0 benchmark roadmap).
10. Bangladesh Supermarket Owners Association (BSOA), 'Annual Report on Supermarket Operations, Wastage, and Modern Trade Dynamics in Bangladesh,' Dhaka, 2023.
11. Food and Agriculture Organization of the United Nations (FAO), 'Food Loss and Waste in Retail Supply Chains in South Asia,' Rome, 2021.